

Non-functional Requirements

 systemdesignschool.io/fundamentals/non-functional-requirements



System design starts with functional requirements: what features to build. Non-functional requirements define how the system behaves: response times, uptime guarantees, and user capacity. These constraints fundamentally shape architectural decisions.

Scale drives complexity

A chat app for 100 users needs basic request-response patterns. The same app for 100 million users requires caching layers, message queues, and distributed databases. Twitter's timeline demonstrates this clearly: simple database queries work for thousands of users, but millions require pre-computed feeds cached in Redis.

The big four constraints

Most systems optimize for four key non-functional requirements: availability (uptime), scalability (handling growth), latency (response speed), and consistency (data accuracy). Each constraint involves trade-offs that influence every architectural choice.

Availability: Staying Online

[Availability](#) measures system uptime as a percentage. 99.9% availability allows 8.77 hours of downtime per year, while 99.99% permits just 52.6 minutes. This difference drives completely different architectural approaches.

Redundancy prevents single points of failure

Production systems use multiple servers across different data centers. When one fails, traffic automatically routes to healthy instances. Netflix runs identical services across three AWS availability zones - if an entire zone fails, users keep streaming without interruption.

Health checks enable fast recovery

Load balancers continuously probe backend servers every few seconds. Failed health checks trigger automatic traffic rerouting within 10-30 seconds. Slack's load balancers detect server failures and redistribute websocket connections before users notice disconnections.

Geographic distribution reduces blast radius

Critical systems replicate across continents. When AWS's us-east-1 region experiences outages, services like Route 53 continue operating from other regions. This isolation prevents single-region failures from causing global outages.

Scalability: Handling Growth

Scalability means maintaining performance as load increases. In practice, scaling out with many small servers typically beats scaling up with larger machines. This approach enables elastic growth and reduces failure impact.

Stateless services scale horizontally

Each server handles requests independently without storing user sessions or temporary data. This design allows adding or removing servers based on demand. Uber's ride-matching service runs hundreds of identical stateless containers that AWS auto-scaling launches during peak hours.

Caching reduces database pressure

Frequently accessed data stays in fast memory instead of slow disk storage. Reddit caches post rankings and user profiles in Redis, handling millions of requests without overwhelming their PostgreSQL database. Cache hit rates above 90% are common for read-heavy workloads.

Async processing smooths traffic spikes

Message queues decouple request handling from background processing. When users upload videos to YouTube, the upload completes immediately while transcoding happens asynchronously. This prevents encoding delays from affecting user experience during traffic surges.

Latency: Speed Matters

[Latency](#) measures response time - the delay between request and response. Users expect sub-second responses for web pages and millisecond responses for API calls. High latency kills user engagement and business metrics.

Geographic proximity reduces network delay

Content Delivery Networks place static assets near users. Cloudflare serves images and CSS files from 270+ global locations, reducing average latency from 200ms to 20ms for distant users. Physical distance between servers and users creates unavoidable delays - light travels at a fixed speed.

Database optimization eliminates bottlenecks

Slow queries kill performance. Instagram optimized their photo feed queries from 1 second to 50ms by adding database indexes and reducing joins. Query profiling reveals which operations consume the most time, enabling targeted optimizations.

Edge computing processes data locally

Instead of sending all requests to central servers, edge nodes handle computations near users. Fastly runs customer code at edge locations, processing API requests without round-trips to origin servers. This approach reduces latency by 10x for globally distributed applications.

Consistency: Data Accuracy

Consistency ensures all system replicas show identical data. In distributed systems, this becomes complex because network partitions and server failures can create temporary disagreements between nodes.

Strong consistency sacrifices availability

Bank account balances require immediate consistency across all servers - users can't withdraw the same \$100 twice. Traditional banks use synchronous replication where all database replicas must confirm writes before transactions complete. This guarantees accuracy but creates single points of failure.

Eventual consistency enables scale

Social media systems tolerate temporary inconsistencies for better performance. When you post on Facebook, different users might see the post at slightly different times as updates propagate across data centers. The system eventually converges to a consistent state within seconds.

Choosing the right consistency model

Most systems mix consistency levels by operation type. Amazon's shopping cart uses eventual consistency (items might briefly disappear then reappear), but payment processing requires strong consistency (charges must be exact). This hybrid approach optimizes for both user experience and data correctness.

Design Patterns for Non-Functional Requirements

Different architectural patterns address specific non-functional requirements. Understanding these relationships helps choose appropriate solutions for system constraints.

Requirement	Core Strategy	Key Technologies	Production Example
Availability	Eliminate single points of failure	Load balancers, multi-region deployment, health checks	Netflix: 3 availability zones per service
Scalability	Scale horizontally with stateless services	Auto-scaling, caching, message queues	Uber: Container orchestration for demand spikes
Latency	Minimize network hops and processing time	CDNs, database indexes, edge computing	Instagram: 50ms photo feed queries
Consistency	Choose appropriate consistency model	Synchronous replication, consensus algorithms	Amazon: Strong for payments, eventual for recommendations

Trade-offs shape architecture

These requirements often conflict. Strong consistency reduces availability during network partitions. Low latency increases complexity and cost. Successful systems prioritize requirements based on business needs rather than trying to optimize everything equally.